

```

26         exit(1);
27     }
28
29     /* Now we can call addvec() just like any other function */
30     addvec(x, y, z, 2);
31     printf("z = [%d %d]\n", z[0], z[1]);
32
33     /* Unload the shared library */
34     if (dlclose(handle) < 0) {
35         fprintf(stderr, "%s\n", dlerror());
36         exit(1);
37     }
38     return 0;
39 }

```

code/link/dll.c

图 7-17 (续)

旁注 共享库和 Java 本地接口

Java 定义了一个标准调用规则，叫做 Java 本地接口 (Java Native Interface, JNI)，它允许 Java 程序调用“本地的”C 和 C++ 函数。JNI 的基本思想是将本地 C 函数(如 foo)编译到一个共享库中(如 foo.so)。当一个正在运行的 Java 程序试图调用函数 foo 时，Java 解释器利用 dlopen 接口(或者与其类似的接口)动态链接和加载 foo.so，然后再调用 foo。

7.12 位置无关代码

共享库的一个主要目的就是允许多个正在运行的进程共享内存中相同的库代码，因而节约宝贵的内存资源。那么，多个进程是如何共享程序的一个副本的呢？一种方法是给每个共享库分配一个事先预备的专用的地址空间片，然后要求加载器总是在这个地址加载共享库。虽然这种方法很简单，但是它也造成了一些严重的问题。它对地址空间的使用效率不高，因为即使一个进程不使用这个库，那部分空间还是会被分配出来。它也难以管理。我们必须保证没有片会重叠。每次当一个库修改了之后，我们必须确认已分配给它的片还适合它的大小。如果不适合了，必须找一个新的片。并且，如果创建了一个新的库，我们还必须为它寻找空间。随着时间的进展，假设在一个系统中有了成百个库和库的各个版本库，就很难避免地址空间分裂成大量小的、未使用而又不再生能使用的小洞。更糟的是，对每个系统而言，库在内存中的分配都是不同的，这就引起了更多令人头痛的管理问题。

要避免这些问题，现代系统以这样一种方式编译共享模块的代码段，使得可以把它们加载到内存的任何位置而无需链接器修改。使用这种方法，无限多个进程可以共享一个共享模块的代码段的单一副本。(当然，每个进程仍然会有它自己的读/写数据块。)

可以加载而无需重定位的代码称为位置无关代码(Position-Independent Code, PIC)。用户对 GCC 使用 -fpic 选项指示 GNU 编译系统生成 PIC 代码。共享库的编译必须总是使用该选项。

在一个 x86-64 系统中，对同一个目标模块中符号的引用是不需要特殊处理使之成为 PIC。可以用 PC 相对寻址来编译这些引用，构造目标文件时由静态链接器重定位。然而，对共享模块定义的外部过程和对全局变量的引用需要一些特殊的技巧，接下来我们会谈到。

1. PIC 数据引用

编译器通过运用以下这个有趣的事实来生成对全局变量的 PIC 引用：无论我们在内存中的何处加载一个目标模块(包括共享目标模块)，数据段与代码段的距离总是保持不变。因此，代码段中任何指令和数据段中任何变量之间的距离都是一个运行时常量，与代码段和数据段的绝对内存位置是无关的。

想要生成对全局变量 PIC 引用的编译器利用了这个事实，它在数据段开始的地方创建了一个表，叫做全局偏移量表(Global Offset Table, GOT)。在 GOT 中，每个被这个目标模块引用的全局数据目标(过程或全局变量)都有一个 8 字节条目。编译器还为 GOT 中每个条目生成一个重定位记录。在加载时，动态链接器会重定位 GOT 中的每个条目，使得它包含目标的正确的绝对地址。每个引用全局目标的目标模块都有自己的 GOT。

图 7-18 展示了示例 libvector.so 共享模块的 GOT。addvec 例程通过 GOT[3]间接地加载全局变量 addcnt 的地址，然后把 addcnt 在内存中加 1。这里的关键思想是对 GOT[3]的 PC 相对引用中的偏移量是一个运行时常量。

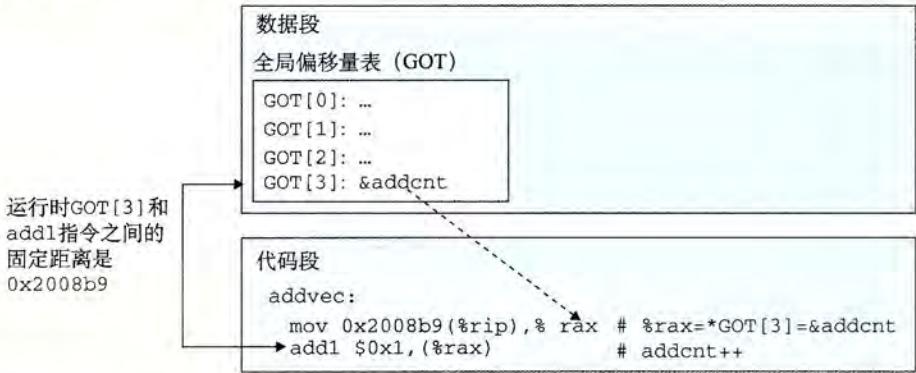


图 7-18 用 GOT 引用全局变量。libvector.so 中的 addvec 例程通过 libvector.so 的 GOT 间接引用了 addcnt

因为 addcnt 是由 libvector.so 模块定义的，编译器可以利用代码段和数据段之间不变的距离，产生对 addcnt 的直接 PC 相对引用，并增加一个重定位，让链接器在构造这个共享模块时解析它。不过，如果 addcnt 是由另一个共享模块定义的，那么就需要通过 GOT 进行间接访问。在这里，编译器选择采用最通用的解决方案，为所有的引用使用 GOT。

2. PIC 函数调用

假设程序调用一个由共享库定义的函数。编译器没有办法预测这个函数的运行时地址，因为定义它的共享模块在运行时可以加载到任意位置。正常的方法是为该引用生成一条重定位记录，然后动态链接器在程序加载的时候再解析它。不过，这种方法并不是 PIC，因为它需要链接器修改调用模块的代码段，GNU 编译系统使用了一种很有趣的技术来解决这个问题，称为延迟绑定(lazy binding)，将过程地址的绑定推迟到第一次调用该过程时。

使用延迟绑定的动机是对于一个像 libc.so 这样的共享库输出的成百上千个函数中，一个典型的应用程序只会使用其中很少的一部分。把函数地址的解析推迟到它实际被调用的地方，能避免动态链接器在加载时进行成百上千个其实并不需要的重定位。第一次调用过程的运行时开销很大，但是其后的每次调用都只会花费一条指令和一个间接的内存引用。

延迟绑定是通过两个数据结构之间简洁但又有些复杂的交互来实现的，这两个数据结

构是：GOT 和过程链接表(Procedure Linkage Table, PLT)。如果一个目标模块调用定义在共享库中的任何函数，那么它就有自己的 GOT 和 PLT。GOT 是数据段的一部分，而 PLT 是代码段的一部分。

图 7-19 展示的是 PLT 和 GOT 如何协作在运行时解析函数的地址。首先，让我们检查一下这两个表的内容。

- 过程链接表(PLT)。PLT 是一个数组，其中每个条目是 16 字节代码。PLT[0]是一个特殊条目，它跳转到动态链接器中。每个被可执行程序调用的库函数都有它自己的 PLT 条目。每个条目都负责调用一个具体的函数。PLT[1](图中未显示)调用系统启动函数(`__libc_start_main`)，它初始化执行环境，调用 `main` 函数并处理其返回值。从 PLT[2]开始的条目调用用户代码调用的函数。在我们的例子中，PLT[2]调用 `addvec`，PLT[3](图中未显示)调用 `printf`。
- 全局偏移量表(GOT)。正如我们看到的，GOT 是一个数组，其中每个条目是 8 字节地址。和 PLT 联合使用时，GOT[0]和 GOT[1]包含动态链接器在解析函数地址时会使用的信息。GOT[2]是动态链接器在 `ld-linux.so` 模块中的入口点。其余的每个条目对应于一个被调用的函数，其地址需要在运行时被解析。每个条目都有一个相匹配的 PLT 条目。例如，GOT[4]和 PLT[2]对应于 `addvec`。初始时，每个 GOT 条目都指向对应 PLT 条目的第二条指令。

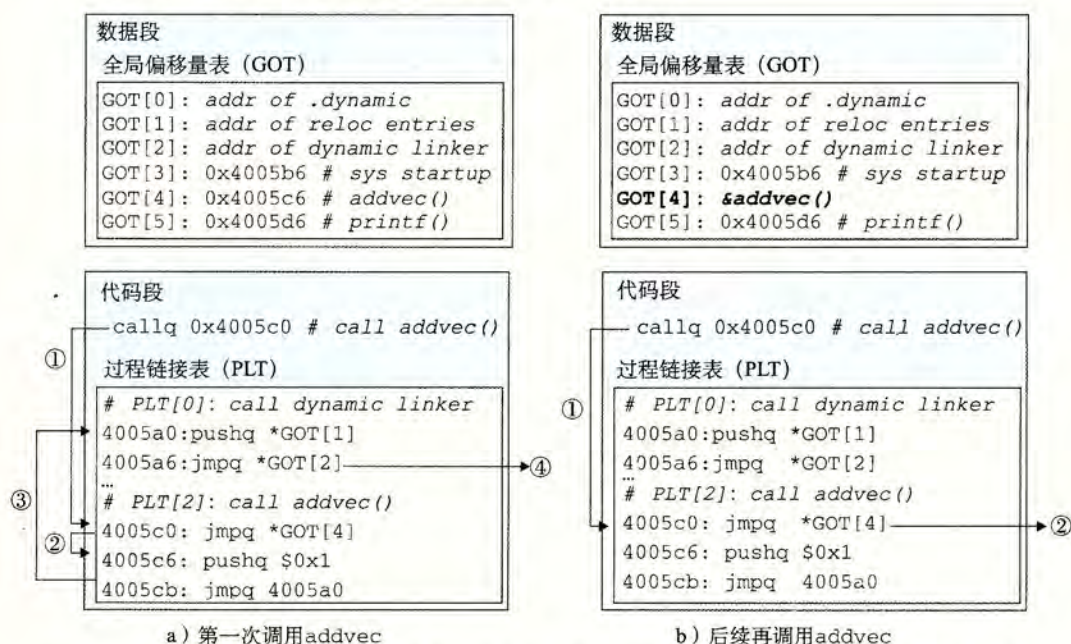


图 7-19 用 PLT 和 GOT 调用外部函数。在第一次调用 `addvec` 时，动态链接器解析它的地址

图 7-19a 展示了 GOT 和 PLT 如何协同工作，在 `addvec` 被第一次调用时，延迟解析它的运行时地址：

- 第 1 步。不直接调用 `addvec`，程序调用进入 PLT[2]，这是 `addvec` 的 PLT 条目。
- 第 2 步。第一条 PLT 指令通过 GOT[4] 进行间接跳转。因为每个 GOT 条目初始时都指向它对应的 PLT 条目的第二条指令，这个间接跳转只是简单地把控制传送回 PLT[2] 中的下一条指令。

- 第 3 步。在把 `addvec` 的 ID(0x1) 压入栈中之后, `PLT[2]` 跳转到 `PLT[0]`。
- 第 4 步。`PLT[0]` 通过 `GOT[1]` 间接地把动态链接器的一个参数压入栈中, 然后通过 `GOT[2]` 间接跳转进动态链接器中。动态链接器使用两个栈条目来确定 `addvec` 的运行时位置, 用这个地址重写 `GOT[4]`, 再把控制传递给 `addvec`。

图 7-19b 给出的是后续再调用 `addvec` 时的控制流:

- 第 1 步。和前面一样, 控制传递到 `PLT[2]`。
- 第 2 步。不过这次通过 `GOT[4]` 的间接跳转会将控制直接转移到 `addvec`。

7.13 库打桩机制

Linux 链接器支持一个很强大的技术, 称为库打桩(library interpositioning), 它允许你截获对共享库函数的调用, 取而代之执行自己的代码。使用打桩机制, 你可以追踪对某个特殊库函数的调用次数, 验证和追踪它的输入和输出值, 或者甚至把它替换成一个完全不同的实现。

下面是它的基本思想: 给定一个需要打桩的目标函数, 创建一个包装函数, 它的原型与目标函数完全一样。使用某种特殊的打桩机制, 你就可以欺骗系统调用包装函数而不是目标函数了。包装函数通常会执行它自己的逻辑, 然后调用目标函数, 再将目标函数的返回值传递给调用者。

打桩可以发生在编译时、链接时或当程序被加载和执行的运行时。要研究这些不同的机制, 我们以图 7-20a 中的示例程序作为运行例子。它调用 C 标准库(`libc.so`)中的 `malloc` 和 `free` 函数。对 `malloc` 的调用从堆中分配一个 32 字节的块, 并返回指向该块的指针。对 `free` 的调用把块还回到堆, 供后续的 `malloc` 调用使用。我们的目标是用打桩来追踪程序运行时对 `malloc` 和 `free` 的调用。

7.13.1 编译时打桩

图 7-20 展示了如何使用 C 预处理器在编译时打桩。`mymalloc.c` 中的包装函数(图 7-20c)调用目标函数, 打印追踪记录, 并返回。本地的 `malloc.h` 头文件(图 7-20b)指示预处理器用对相应包装函数的调用替换掉对目标函数的调用。像下面这样编译和链接这个程序:

```
linux> gcc -DCOMPILETIME -c mymalloc.c
linux> gcc -I. -o intc int.c mymalloc.o
```

由于有 `-I.` 参数, 所以会进行打桩, 它告诉 C 预处理器在搜索通常的系统目录之前, 先在当前目录中查找 `malloc.h`。注意, `mymalloc.c` 中的包装函数是使用标准 `malloc.h` 头文件编译的。

运行这个程序会得到如下的追踪信息:

```
linux> ./intc
malloc(32)=0x9ee010
free(0x9ee010)
```

7.13.2 链接时打桩

Linux 静态链接器支持用 `-wrap f` 标志进行链接时打桩。这个标志告诉链接器, 把对符号 `f` 的引用解析成 `__wrap_f` (前缀是两个下划线), 还要把对符号 `__real_f` (前缀是两个下划线) 的引用解析为 `f`。图 7-21 给出我们示例程序的包装函数。

code/link/interpose/int.c

```

1  #include <stdio.h>
2  #include <malloc.h>
3
4  int main()
5  {
6      int *p = malloc(32);
7      free(p);
8      return(0);
9  }

```

code/link/interpose/int.c

a) 示例程序 int.c

code/link/interpose/malloc.h

```

1  #define malloc(size) mymalloc(size)
2  #define free(ptr) myfree(ptr)
3
4  void *mymalloc(size_t size);
5  void myfree(void *ptr);

```

code/link/interpose/malloc.h

b) 本地 malloc.h 文件

code/link/interpose/mymalloc.c

```

1  #ifdef COMPILETIME
2  #include <stdio.h>
3  #include <malloc.h>
4
5  /* malloc wrapper function */
6  void *mymalloc(size_t size)
7  {
8      void *ptr = malloc(size);
9      printf("malloc(%d)=%p\n",
10             (int)size, ptr);
11     return ptr;
12 }
13
14 /* free wrapper function */
15 void myfree(void *ptr)
16 {
17     free(ptr);
18     printf("free(%p)\n", ptr);
19 }
20 #endif

```

code/link/interpose/mymalloc.c

c) mymalloc.c 中的包装函数

图 7-20 用 C 预处理器进行编译时打桩

用下述方法把这些源文件编译成可重定位目标文件：

```

linux> gcc -DLINKTIME -c mymalloc.c
linux> gcc -c int.c

```

然后把目标文件链接成可执行文件：

```

linux> gcc -Wl,--wrap,malloc -Wl,--wrap,free -o intl int.o mymalloc.o

```

-Wl,option 标志把 option 传递给链接器。option 中的每个逗号都要替换为一个空

格。所以`-Wl,--wrap,malloc`就把`--wrap malloc`传递给链接器，以类似的方式传递`-Wl,--wrap,free`。

```

code/link/interpose/mymalloc.c
1  #ifdef LINKTIME
2  #include <stdio.h>
3
4  void *__real_malloc(size_t size);
5  void __real_free(void *ptr);
6
7  /* malloc wrapper function */
8  void *__wrap_malloc(size_t size)
9  {
10     void *ptr = __real_malloc(size); /* Call libc malloc */
11     printf("malloc(%d) = %p\n", (int)size, ptr);
12     return ptr;
13 }
14
15 /* free wrapper function */
16 void __wrap_free(void *ptr)
17 {
18     __real_free(ptr); /* Call libc free */
19     printf("free(%p)\n", ptr);
20 }
21 #endif
code/link/interpose/mymalloc.c

```

图 7-21 用`--wrap`标志进行链接时打桩

运行该程序会得到如下追踪信息：

```

linux> ./intl
malloc(32) = 0x18cf010
free(0x18cf010)

```

7.13.3 运行时打桩

编译时打桩需要能够访问程序的源代码，链接时打桩需要能够访问程序的可重定位对象文件。不过，有一种机制能够在运行时打桩，它只需要能够访问可执行目标文件。这个很厉害的机制基于动态链接器的`LD_PRELOAD`环境变量。

如果`LD_PRELOAD`环境变量被设置为一个共享库路径名的列表（以空格或分号分隔），那么当你加载和执行一个程序，需要解析未定义的引用时，动态链接器（`LD-LINUX.SO`）会先搜索`LD_PRELOAD`库，然后才搜索任何其他的库。有了这个机制，当你加载和执行任意可执行文件时，可以对任何共享库中的任何函数打桩，包括`libc.so`。

图 7-22 展示了`malloc`和`free`的包装函数。每个包装函数中，对`dlsym`的调用返回指向目标`libc`函数的指针。然后包装函数调用目标函数，打印追踪记录，再返回。

下面是如何构建包含这些包装函数的共享库的方法：

```
linux> gcc -DRUNTIME -shared -fpic -o mymalloc.so mymalloc.c -ldl
```

这是如何编译主程序：

```
linux> gcc -o intr int.c
```

```

1  #ifdef RUNTIME
2  #define _GNU_SOURCE
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <dlfcn.h>
6
7  /* malloc wrapper function */
8  void *malloc(size_t size)
9  {
10     void *(*mallocp)(size_t size);
11     char *error;
12
13     mallocp = dlsym(RTLD_NEXT, "malloc"); /* Get address of libc malloc */
14     if ((error = dlerror()) != NULL) {
15         fputs(error, stderr);
16         exit(1);
17     }
18     char *ptr = mallocp(size); /* Call libc malloc */
19     printf("malloc(%d) = %p\n", (int)size, ptr);
20     return ptr;
21 }
22
23 /* free wrapper function */
24 void free(void *ptr)
25 {
26     void (*freep)(void *) = NULL;
27     char *error;
28
29     if (!ptr)
30         return;
31
32     freep = dlsym(RTLD_NEXT, "free"); /* Get address of libc free */
33     if ((error = dlerror()) != NULL) {
34         fputs(error, stderr);
35         exit(1);
36     }
37     freep(ptr); /* Call libc free */
38     printf("free(%p)\n", ptr);
39 }
40 #endif

```

code/link/interpose/mymalloc.c

图 7-22 用 LD_PRELOAD 进行运行时打桩

下面是如何从 bash shell 中运行这个程序[⊖]：

```

linux> LD_PRELOAD="./mymalloc.so" ./intr
malloc(32) = 0x1bf7010
free(0x1bf7010)

```

⊖ 如果你不知道运行的 shell 是哪一种，在命令行上输入 printenv SHELL。

下面是如何在 `csch` 或 `tcsh` 中运行这个程序：

```
linux> (setenv LD_PRELOAD "./mymalloc.so"; ./intr; unsetenv LD_PRELOAD)
malloc(32) = 0x2157010
free(0x2157010)
```

请注意，你可以用 `LD_PRELOAD` 对任何可执行程序库函数调用打桩！

```
linux> LD_PRELOAD="./mymalloc.so" /usr/bin/uptime
malloc(568) = 0x21bb010
free(0x21bb010)
malloc(15) = 0x21bb010
malloc(568) = 0x21bb030
malloc(2255) = 0x21bb270
free(0x21bb030)
malloc(20) = 0x21bb030
malloc(20) = 0x21bb050
malloc(20) = 0x21bb070
malloc(20) = 0x21bb090
malloc(20) = 0x21bb0b0
malloc(384) = 0x21bb0d0
20:47:36 up 85 days, 6:04, 1 user, load average: 0.10, 0.04, 0.05
```

7.14 处理目标文件的工具

在 Linux 系统中有大量可用的工具可以帮助你理解和处理目标文件。特别地，GNU `binutils` 包尤其有帮助，而且可以运行在每个 Linux 平台上。

- `AR`：创建静态库，插入、删除、列出和提取成员。
- `STRINGS`：列出一个目标文件中所有可打印的字符串。
- `STRIP`：从目标文件中删除符号表信息。
- `NM`：列出一个目标文件的符号表中定义的符号。
- `SIZE`：列出目标文件中节的名字和大小。
- `READELF`：显示一个目标文件的完整结构，包括 ELF 头中编码的所有信息。包含 `SIZE` 和 `NM` 的功能。
- `OBJDUMP`：所有二进制工具之母。能够显示一个目标文件中所有的信息。它最大的作用是反汇编 `.text` 节中的二进制指令。

Linux 系统为操作共享库还提供了 `LDD` 程序：

- `LDD`：列出一个可执行文件在运行时所需要的共享库。

7.15 小结

链接可以在编译时由静态编译器来完成，也可以在加载时和运行时由动态链接器来完成。链接器处理称为目标文件的二进制文件，它有 3 种不同的形式：可重定位的、可执行的和共享的。可重定位的目标文件由静态链接器合并成一个可执行的目标文件，它可以加载到内存中并执行。共享目标文件（共享库）是在运行时由动态链接器链接和加载的，或者隐含地在调用程序被加载和开始执行时，或者根据需要在程序调用 `dlopen` 库的函数时。

链接器的两个主要任务是符号解析和重定位，符号解析将目标文件中的每个全局符号都绑定到一个唯一的定义，而重定位确定每个符号的最终内存地址，并修改对那些目标的引用。